МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

КУРСОВАЯ РАБОТА

по дисциплине «Операционные системы специального назначения»

Тема: Исследование процессов и планирования заданий в операционной системе Linux

Студент гр. 3364 _____ Васильев А.П.

Преподаватель _____ Корчагин П.В.

Санкт-Петербург

2025

ЗАДАНИЕ
НА КУРСОВУЮ РАБОТУ

Студент Васильев А.П.

Группа 3364

Тема работы: Исследование процессов и планирования заданий в операционной системе Linux

Содержание пояснительной записки:

Введение, теоретические основы процессов в Linux, понятие процесса и потока, жизненный цикл процесса, планировщик задач, практическое исследование управления процессами, работа с сигналами, инструменты командной строки для мониторинга, управление заданиями, экспериментальная часть, эксперимент с nice и renice, анализ процесса через /proc, анализ системы при помощи утилиты atop, заключение, список использованных источников.

Предполагаемый объем пояснительной записки:

Не менее 40 страниц.

Дата выдачи задания: 20.11.2025

Дата сдачи реферата: 15.12.2025

Дата защиты реферата: 15.12.2023

Студент _____ Васильев А.П.

Преподаватель _____ Корчагин П.В.

АННОТАЦИЯ

Данная курсовая работа посвящена исследованию модели процессов в операционной системе Linux на примере Astra Linux Special Edition. В работе проводится теоретический анализ жизненного цикла и состояний процесса, механизмов планирования и системных вызовов. Практическая часть включает использование командных утилит (ps, top, /proc и других) и эксперименты по управлению процессами.

SUMMARY

This course work is devoted to the study of the process model in the Linux operating system using the example of Astra Linux Special Edition. The paper provides a theoretical analysis of the life cycle and states of the process, planning mechanisms and system calls. The practical part includes the use of command utilities (ps, top, /proc and others) and experiments on process management.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЦЕССОВ В LINUX 7

1.1. Понятие процесса и потока (thread) 7

1.2. Жизненный цикл процесса 10

1.3. Планировщик задач (Scheduler) 15

2. Практическое исследование управлениями процессами 20

2.1. Работа с сигналами 20

2.2. Инструменты командной строки для мониторинга 25

2.3. Управление заданиями (Job Control) 31

3. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ 34

3.1. Эксперимент с nice и renice 34

3.2. Анализ процесса через /proc 36

3.2. Анализ системы при помощи утилиты atop 39

ЗАКЛЮЧЕНИЕ 42

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 43

ВВЕДЕНИЕ

Современные операционные системы представляют собой сложные программные комплексы, обеспечивающие эффективное распределение вычислительных ресурсов и параллельное выполнение множества задач. Одним из ключевых элементов любой многозадачной операционной системы является модель процессов – базовая единица выполнения программного кода, через которую осуществляется организация вычислений, управление ресурсами и взаимодействие между компонентами программного обеспечения. Исследование того, как операционная система, в частности Linux, реализует эти механизмы через модель процессов и их планировщик, составляет основное содержание данной работы.

Актуальность данного исследования обусловлена повсеместным распространением операционных систем семейства Linux, лежащих в основе серверной инфраструктуры, мобильных устройств и встроенных систем. Особую значимость работа приобретает в контексте операционных систем специального назначения, к которым предъявляются жёсткие требования по безопасности, отказоустойчивости и контролю.

Объектом исследования в данной работе является операционная система Astra Linux Special Edition – российская операционная система специального назначения (ОССН) с встроенными средствами защиты информации, разработанная для работы с данными, имеющими ограниченный доступ.

Предметом исследования в данной курсовой работе является модель процессов в Linux и механизмы их управления, включая способы создания и завершения процессов, особенности их взаимодействия, работу системных вызовов и алгоритмов планировщика.

Целью курсовой работы является исследование модели процессов в Linux, их жизненного цикла, а также способов управления и планирования.

Перед выполнением курсовой работы были поставлены некоторые **задачи**, а именно:

1) Изучить теоретические основы понятия «процесс» в Linux;

2) Проанализировать жизненный цикл процесса и его состояния;

3) Исследовать системные вызовы для управления процессами;

4) Практически изучить инструменты командной строки для мониторинга и управления процессами;

5) Исследовать механизмы планирования процессов (scheduler);

6) Создать практические примеры (демоны, фоновые задания, параллельное выполнение).

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЦЕССОВ В LINUX

1.1. Понятие процесса и потока (thread)

В операционной системе Linux процессом называют изолированный экземпляр выполняемой программы. Когда пользователь запускает приложение — будь то текстовый редактор, веб-браузер, серверная служба или простейшая консольная утилита — ядро создаёт процесс, которому выделяется собственное адресное пространство, набор ресурсов и запись в глобальной таблице процессов. Процесс можно рассматривать как сущность, объединяющую в себе исполняемый код, динамически выделяемую память, открытые файлы, состояние регистров процессора и прочие параметры, необходимые для выполнения программы.

Процесс — это не просто «запущенный код», а структурированная единица выполнения, полностью управляемая ядром. Linux обеспечивает изоляцию процессов: один процесс не может напрямую получить доступ к памяти другого (за исключением специально организованных механизмов — shared memory, mmap и т.п.), что является ключевым принципом безопасности и стабильности ОС.

Каждый процесс в Linux обладает набором характеристик и ресурсов, которые ядро хранит в структуре, известной как task_struct. Можно рассматривать эту структуру как «паспорт» процесса, который ядро заводит при его рождении и аннулирует при завершении. Основные компоненты структуры процесса включают:

PID (Process ID) – Это уникальный числовой идентификатор процесса, присваиваемый при его создании. В Linux данные идентификаторы имеют ограниченный диапазон – по умолчанию до 32768, после исчерпания которого нумерация начинается заново. PID используется ядром и пользовательскими программами для управления процессом: отправки сигналов, отслеживания состояния, получения статистики.

PPID (Parent Process ID) – идентификатор процесса-родителя, который создал данный процесс. Все процессы, за исключением init/systemd (PID = 1), имеют родительский процесс. Это образует древовидную структуру процессов – так называемое «дерево процессов».

Адресное пространство – это диапазон виртуальных адресов, который операционная система выделяет каждому процессу. Виртуальное адресное пространство разделяется на несколько логических сегментов:

- Сегмент кода (text) — область, доступная только для чтения и выполнения, содержащая инструкции программы;
- Сегмент данных (data) — область для инициализированных статических и глобальных переменных (чтение/запись);
- Сегмент BSS — область для неинициализированных глобальных и статических переменных, которые автоматически обнуляются при запуске;
- Куча (heap) — область для динамического выделения памяти во время выполнения программы;
- Стек (stack) — область для хранения локальных переменных и управления вызовами функций.

Таблица открытых файлов является одной из ключевых структур данных, ассоциированных с каждым процессом. Её можно представить как массив указателей, индексированный целыми числами — файловыми дескрипторами. Каждый открытый процессом ресурс ввода-вывода (обычный файл, каталог, сокет, канал, устройство) представлен в этой таблице записью. По умолчанию процесс наследует от родителя 3 стандартных дескриптора:

- 0 (STDIN_FILENO) — стандартный ввод (stdin), обычно ассоциирован с клавиатурой или перенаправлен из файла/пайпа.
- 1 (STDOUT_FILENO) — стандартный вывод (stdout), обычно ассоциирован с терминалом или перенаправлен в файл.

- 2 (STDERR_FILENO) — стандартный вывод ошибок (stderr), по умолчанию также направлен на терминал, но логически отделён от stdout для фильтрации сообщений об ошибках.

Контекст выполнения — это "мгновенный снимок" состояния процессора и связанных структур данных, уникальный для каждого процесса в момент его выполнения. Это ключевая концепция для реализации многозадачности, безопасности и отказоустойчивости системы, поскольку позволяет сохранить состояние прерванного процесса и позже восстановить его выполнение точно с того же места. Ключевым элементом этого снимка являются регистры центрального процессора (CPU):

- Программный счетчик (RIP/IP) хранит адрес следующей выполняемой инструкции.
- Указатель стека (RSP/SP) ссылается на вершину стека процесса.
- Регистры общего назначения (RAX, RBX, RCX и др.) содержат промежуточные данные вычислений.
- Регистр состояния (RFLAGS/EFLAGS) хранит флаги результатов операций (например, был ли результат нулевым).
- Управляющие регистры, такие как CR3, который определяет виртуальное адресное пространство процесса, обеспечивая его изоляцию.

Поток (thread) — объекты в составе процесса, которые отвечают за выполнение кода, разделяющие с другими потоками того же процесса адресное пространство, открытые файлы и другие ресурсы.

Несмотря на то что поток выполняется внутри процесса, концепции у этих двух явлений разные. Процесс предназначен для группировки родственных ресурсов. Поток представляет собой набор объектов, которые выполняются в определенном порядке. При этом одновременно внутри одной и той же среды процесса параллельно может исполняться несколько независимых программ. В

этом случае потоки будут совместно использовать, например, открытые файлы и адресное пространство.

В Linux реализация потоков тесно связана с концепцией легковесных процессов (Lightweight Processes, LWP). Каждому потоку в пользовательском пространстве соответствует один LWP в ядре. LWP представляет собой сущность, которую планировщик ядра видит как отдельную задачу для планирования, но при этом несколько LWP могут разделять ресурсы одного процесса. В Linux потоки (нити) реализованы через модель 1:1, где каждому потоку пользовательского пространства соответствует собственный объект ядра — легковесный процесс (Lightweight Process, LWP), который является записью в таблице процессов (структура task_struct), разделяющей с другими LWP одного процесса адресное пространство и ресурсы. Таким образом, для планировщика ядра нет разницы между процессом и потоком — всё это планируемые задачи, различающиеся только степенью разделения ресурсов, заданной при создании через вызов clone().

1.2. Жизненный цикл процесса

Жизненный цикл процесса в Linux — это последовательность состояний, через которые проходит процесс от момента создания до завершения. Ядро управляет этими состояниями, реагируя на события, связанные с планированием, ожиданием ресурсов, вводом-выводом, сигналами и завершением работы. Управление этим циклом — одна из основных функций ОС, обеспечивающая многозадачность, эффективное использование ресурсов и корректное завершение работы программ.

Состояние процесса — это атрибут, определяющий, как планировщик ядра взаимодействует с задачей. В Linux каждое состояние процесса отражает его текущее положение в системе и то, какие действия могут быть выполнены с процессом в данный момент. Основные состояния следующие:

R (Running или Runnable) — процесс либо в данный момент выполняется на ядре процессора, либо находится в очереди готовых к выполнению (runnable) задач и ожидает выделения ему процессорного времени. Большинство процессов проводят в этом состоянии лишь малую часть своего времени.

S (Sleeping, Interruptible Sleep) — процесс приостановил своё выполнение и ожидает наступления какого-либо события. Это может быть ожидание данных от пользователя (чтение с терминала), завершение операции ввода-вывода с диска, освобождение системного ресурса (например, блокировки) или поступление сигнала. Это состояние является прерываемым — если процессу в состоянии S поступит сигнал, он будет «разбужен» и обработает его.

D (Uninterruptible Sleep, или Disk Sleep) — процесс находится в состоянии непрерываемого сна. Обычно это связано с ожиданием завершения низкоуровневой операции ввода-вывода (например, прямого обращения к диску). Критически важно, что процесс в состоянии D не может быть завершён или прерван даже сигналом SIGKILL. Это гарантирует целостность данных и корректность работы драйверов устройств, но может привести к ситуации, когда процесс «зависает» до устранения аппаратной проблемы.

Z (Zombie) — «зомби»-процесс. Это состояние возникает, когда процесс завершил своё выполнение (вызвав exit()), но его запись в системной таблице процессов еще не удалена. Это происходит потому, что родительский процесс пока не прочитал код завершения своего «потомка» с помощью системного вызова wait() (или его вариантов). «Зомби» не потребляет вычислительных ресурсов (памяти, CPU), но занимает запись в таблице процессов. Длительное существование множества «зомби» может указывать на ошибку в программе-родителе, которая не обрабатывает завершение дочерних процессов.

T (Stopped) — выполнение процесса было явно приостановлено (остановлено) сигналом. Чаще всего это происходит при отправке сигнала SIGSTOP или при нажатии комбинации клавиш Ctrl+Z в терминале (что генерирует сигнал SIGTSTP). Возобновить выполнение можно, отправив сигнал SIGCONT.

X (Dead) — завершённый процесс, готовый к удалению из системы. Это состояние очень кратковременное и обычно не отображается в утилитах мониторинга.

Диаграмма переходов между состояниями процессов (R, S, T, D, Z, X) схематично представлена на рисунке 1:

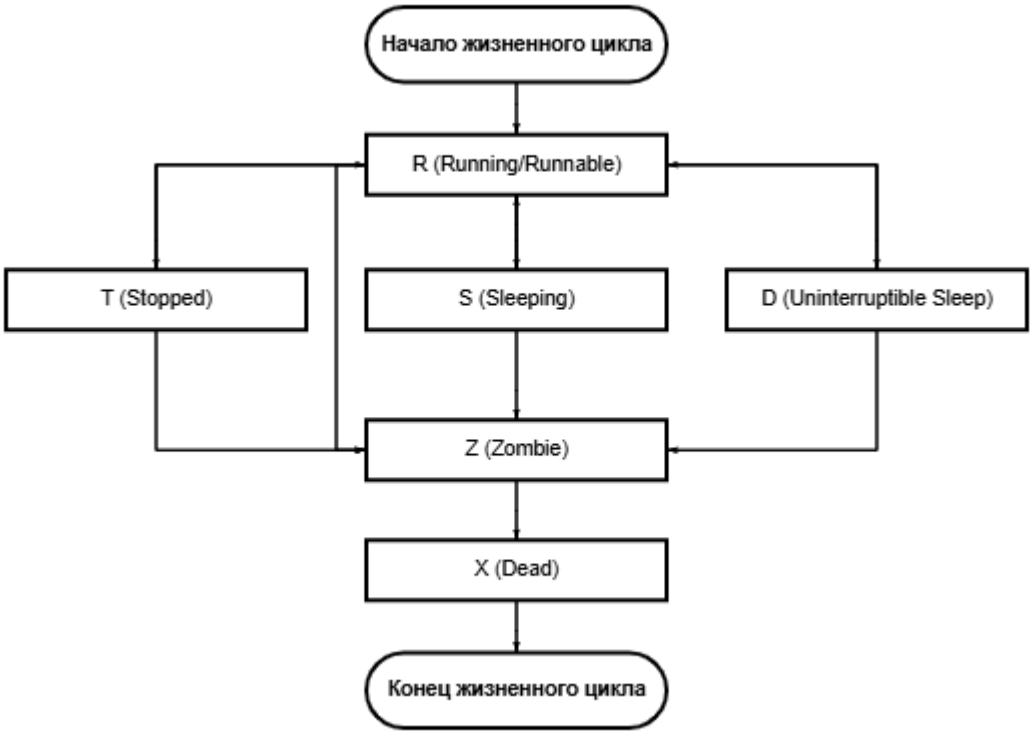

Рисунок 1 – Диаграмма состояний процесса

Создание процессов — важный механизм UNIX-подобных систем. В Linux для этого существуют несколько системных вызовов, каждый со своими особенностями.

Системный вызов fork() — это классический и наиболее часто используемый способ. Он создаёт новый процесс-потомок, который является почти полной копией родителя: наследует код, данные, стек, кучу, таблицу файловых дескрипторов и окружение. После успешного вызова fork() оба процесса продолжают выполнение с одной и той же инструкции, следующей за вызовом. Отличить их можно по возвращаемому значению: в родительский процесс возвращается PID потомка, а в потомка возвращается 0.

Механизм Copy-on-Write (CoW, «копирование при записи») — ключевая оптимизация, применяемая при fork(). Вместо немедленного физического копирования всей памяти родителя (что было бы крайне затратно), ядро настраивает страницы памяти обоих процессов как доступные только для чтения и отображает их на одни и те же физические фреймы. В момент, когда любой из процессов (родитель или потомок) пытается записать в такую страницу, возникает исключение (page fault). Обработчик исключения в ядре прозрачно для процесса создаёт отдельную физическую копию этой страницы только для него. CoW значительно ускоряет создание процессов и экономит память, особенно для программ, которые после fork() сразу вызывают exec() и не модифицируют унаследованные данные.

Системный вызов vfork() — вариант, созданный для случаев, когда fork() с CoW всё ещё считался недостаточно эффективным. vfork() создаёт новый поток выполнения, но без создания нового адресного пространства. Дочерний «процесс» временно работает в адресном пространстве родителя, блокируя его выполнение, пока не вызовет exec() или exit(). Из-за своей опасности и специфичности vfork() считается устаревшим.

Системный вызов clone() — это низкоуровневый и гибкий интерфейс, лежащий в основе как fork(), так и механизма создания потоков pthread_create(). Программист может указать набор флагов (например: CLONE_VM, CLONE_FS, CLONE_FILES), определяющих, какие именно ресурсы (виртуальная память, информация о файловой системе, таблица дескрипторов) будут разделяться между родителем и потомком. Таким образом,

задавая различные параметры clone(), можно создать как полностью изолированный процесс, так и поток, разделяющий с родителем почти все ресурсы.

Завершение процесса с помощью exit(). Процесс завершает своё выполнение следующими основными путями:

1. Нормальное завершение**:** Процесс вызывает системный вызов exit() (или функцию _exit() для немедленного выхода) либо возвращает управление из функции main().

2. Завершение по сигналу: Процесс получает сигнал, действие по умолчанию для которого — завершение (например: SIGTERM, SIGKILL или SIGSEGV).

Последовательность действий при завершении процесса с помощью exit():

1. Закрытие всех файловых дескрипторов;
2. Освобождение динамически выделенной памяти;
3. Уведомление родительского процесса;
4. Перевод процесса в состояние Z (Zombie);
5. Окончательное удаление записи после вызова родителем wait().

Таким образом, завершение процесса — это строгая последовательность действий, гарантирующая корректное освобождение ресурсов и предотвращение утечек.

Особенности для Astra Linux Special Edition: В ОС специального назначения контроль над завершением процессов также является элементом политики безопасности. Администратор может настраивать политики обработки сигналов, аудировать события завершения процессов (особенно критически важных служб) и использовать механизмы контроля целостности (например, запрет на отправку SIGKILL определённым системным демонам), чтобы предотвратить несанкционированное вмешательство или случайный сбой сервисов.

1.3. Планировщик задач (Scheduler)

Планировщик задач является одним из ключевых компонентов ядра Linux, обеспечивающим многозадачность в системе. Его основная задача — распределение процессорного времени между процессами и потоками таким образом, чтобы система оставалась отзывчивой, а ресурсы CPU использовались максимально эффективно. Без планировщика современная операционная система не могла бы поддерживать тысячи параллельно работающих процессов, взаимодействующих в реальном времени.

В условиях многозадачной среды множество процессов и потоков конкурируют за ограниченный набор аппаратных ресурсов — прежде всего, за процессоры. Планировщик определяет:

1. Какой процесс будет выполняться следующим;
2. Как долго он может работать перед переключением на другой;
3. Какие процессы получают больший приоритет, например интерактивные;
4. Как обрабатывать фоновые и вычислительные задачи;
5. Как гарантировать справедливость распределения CPU между пользователями и службами.

В контексте операционных систем специального назначения, таких как Astra Linux Special Edition, роль планировщика расширяется задачами обеспечения безопасности. Планировщик должен гарантировать предсказуемое время отклика критически важных системных служб и, в соответствии с политиками безопасности, изолировать выполнение процессов различного уровня доверия, управляя их доступом к вычислительным ресурсам.

За время развития ядра Linux планировщик несколько раз кардинально менялся, чтобы улучшить масштабируемость, справедливость и предсказуемость работы. Основные этапы можно разделить на четыре поколения.

Планировщик O(n)

Использовался приблизительно до версии Linux 2.4.

Его принцип работы заключался в том, что при выборе следующего процесса к выполнению планировщик просматривал всю очередь процессов, оценивая их приоритеты.

Недостатков является алгоритмическая сложность O(n), где n — количество процессов. При большой загрузке системы (сотни процессов) такое линейное сканирование становилось узким местом, потребляя значительные ресурсы CPU. Планировщик также плохо масштабировался на многопроцессорных системах. То есть, этот планировщик хорошо работал на ранних одноядерных системах, но не подходил для серверов и больших многопроцессорных машин.

Планировщик O(1)

Введён в ядре Linux 2.6 и стал существенным шагом вперёд.

Название отражает главное отличие, по сравнению с предыдущим планировщиком – что время выбора следующей задачи не зависит от количества процессов, а является константным.

Принцип работы следующий: использовалась структура из двух массивов (очередей) для каждого процессорного ядра: активная и неактивная. Каждая очередь состояла из 140 списков (по числу возможных значений приоритета). Планировщик выбирал задачу из списка с наивысшим приоритетом в активной очереди. После исчерпания кванта времени (т.е. фиксированного промежутка процессорного времени) задача перемещалась в неактивную очередь. Когда активная очередь опустеет, очереди менялись местами. Это обеспечивало сложность O(1).

Преимуществом являлась высокая производительность при большом количестве процессов, а также значительное снижение накладных расходов на переключение задач. Но несмотря на скорость, алгоритм был сложен в поддержке и мог вести себя несправедливо по отношению к интерактивным задачам, требуя эвристик (т.е. практических методов, подходов) для их

«разгона». Эти эвристики сами по себе были сложными и не всегда предсказуемыми. Это и стало причиной дальнейшей эволюции.

CFS (Completely Fair Scheduler)

Представлен в Linux 2.6.23 и стал основой планирования в Linux на протяжении почти 16 лет.

CFS был создан как ответ на недостатки O(1). Его философия — не просто выделять кванты времени, а обеспечивать процессам справедливую долю процессорного времени, в зависимости от приоритета процесса.

Ключевая абстракция — виртуальное время (vruntime): каждой задаче ведётся учёт vruntime — нормализованного времени, которое она фактически провела на CPU. Чем выше приоритет задачи, тем медленнее для неё течёт виртуальное время. Простыми словами, vruntime можно представить не как виртуальное время, а как «счётчик очков». Важные процессы набирают эти очки медленнее, чем менее важные. Главным правилом является выполнение приоритетного процесса, то есть того процесса, у которого меньше тех самых очков.

Структура данных — красно-чёрное дерево: готовые к выполнению задачи хранятся в сбалансированном бинарном дереве, отсортированном по vruntime. Планировщик всегда выбирает для выполнения задачу с наименьшим vruntime (ту, которая меньше всех набрала виртуального времени).

Принцип работы: CFS стремится к идеалу, где в любой момент времени vruntime всех задач равны. На практике он выбирает задачу с минимальным vruntime и выполняет её до тех пор, пока её vruntime не превысит значение следующей в дереве. CFS не использует фиксированные кванты времени; длительность выполнения определяется разницей в vruntime.

Преимуществами является высокая степень справедливости, предсказуемость, хорошая масштабируемость для многопроцессорных систем и интуитивно понятное поведение. CFS является планировщиком по умолчанию

для обычных задач в Astra Linux Special Edition. Однако CFS имеет и слабые стороны, например невозможность правильно работать с процессами, которые требуют время процессора, как можно скорее.

EEVDF (Earliest Eligible Virtual Deadline First)

Новый планировщик ядра Linux, включён по умолчанию с версии 6.6, полностью заменил CFS.

EEVDF использует концепцию виртуальных дедлайнов: помимо vruntime, для каждой задачи вычисляется момент дедлайна — виртуальное время, к которому её необходимо запланировать для соблюдения справедливости. Задача считается «пригодной» к выполнению, если её vruntime достиг определённого порога, гарантирующего, что она уже получила свою минимальную долю процессорного времени. Отсюда и слово Elgibility – пригодность.

Принцип заключается в том, что алгоритм выбирает для выполнения не просто задачу с минимальным vruntime, а задачу с наиболее ранним дедлайном, но только среди тех, которые уже стали «пригодными». Это сочетает в себе преимущества справедливости (за счёт vruntime) и строгого учёта временных ограничений (за счёт дедлайна).

Преимущества перед CFS:

Более точное и предсказуемое соблюдение пропорций распределения CPU, особенно в условиях смешанной нагрузки (интерактивные + CPU-интенсивные задачи). Устранение «пограничных» случаев CFS, то есть когда задачи могли получать больше или меньше времени, чем идеально справедливая доля. А также плюсом является более простая логика и потенциально лучшая производительность.

Эволюция планировщиков представлена на рисунке 2:

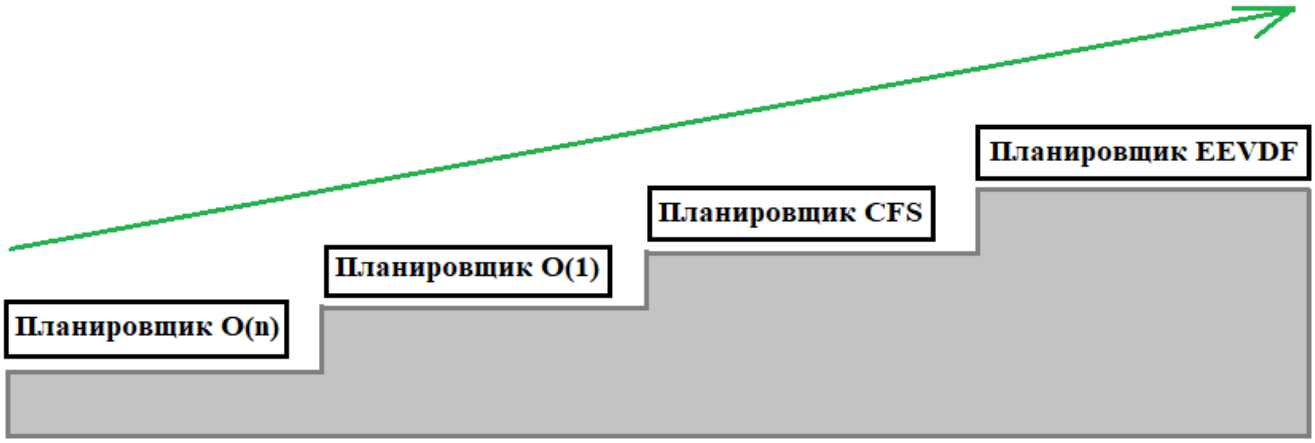

Рисунок 2 – Эволюция планировщиков Linux

Подводя итоги хочется сказать, что Эволюция планировщика Linux от O(n) к EEVDF отражает путь от простого распределения времени к построению сложных математических моделей, гарантирующих справедливость, эффективность и низкую задержку в разных рабочих нагрузках. Современные планировщики, такие как CFS, являются не просто диспетчерами очередей, а сложными системами, балансирующими противоречивые требования тысяч одновременно работающих задач. Для ОССН стабильность и предсказуемость работы планировщика — критически важное свойство, обеспечивающее выполнение требований к безопасности.

2. ПРАКТИЧЕСКОЕ ИССЛЕДОВАНИЕ УПРАВЛЕНИЯ ПРОЦЕССАМИ

2.1. Работа с сигналами

Сигналы (signals) — это механизм асинхронного уведомления процессов о возникновении определённых событий. В Linux сигналы используются для управления процессами, обработки исключительных ситуаций и межпроцессного взаимодействия. Каждый сигнал имеет уникальный номер и символическое имя, начинающееся с префикса SIG.

Утилита kill позволяет отправлять сигналы процессам из командной строки. Несмотря на название, она используется не только для завершения процессов, но и для отправки любых сигналов. Её синтаксис выглядит следующим образом: *kill [-сигнал] PID*.

Для демонстрации работы сигналов, мною был написан небольшой скрипт, который демонстрирует работу процесса и выводит его PID (рисунки 3-4):

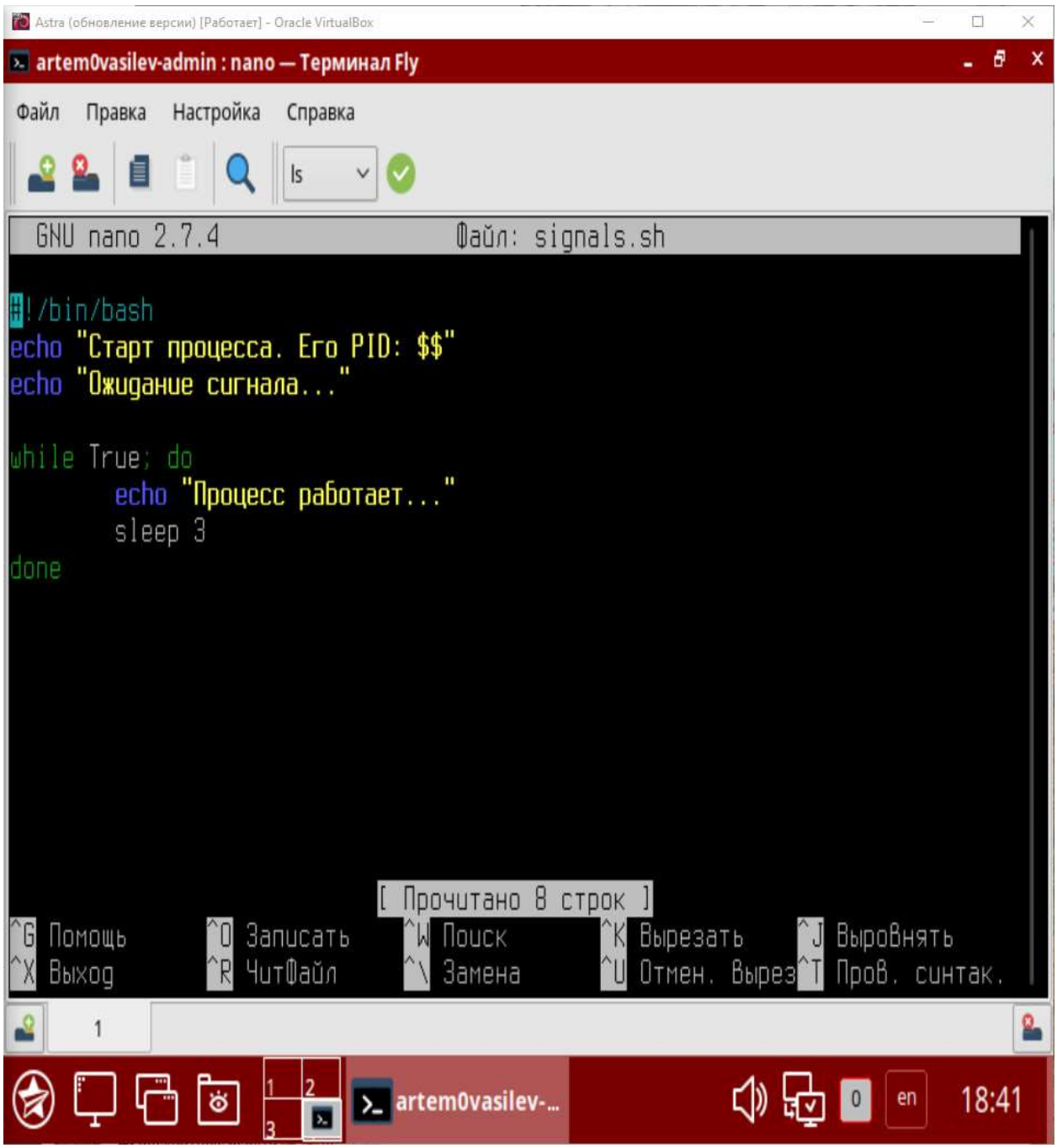

Рисунок 3 – Скрипт для работы с сигналами

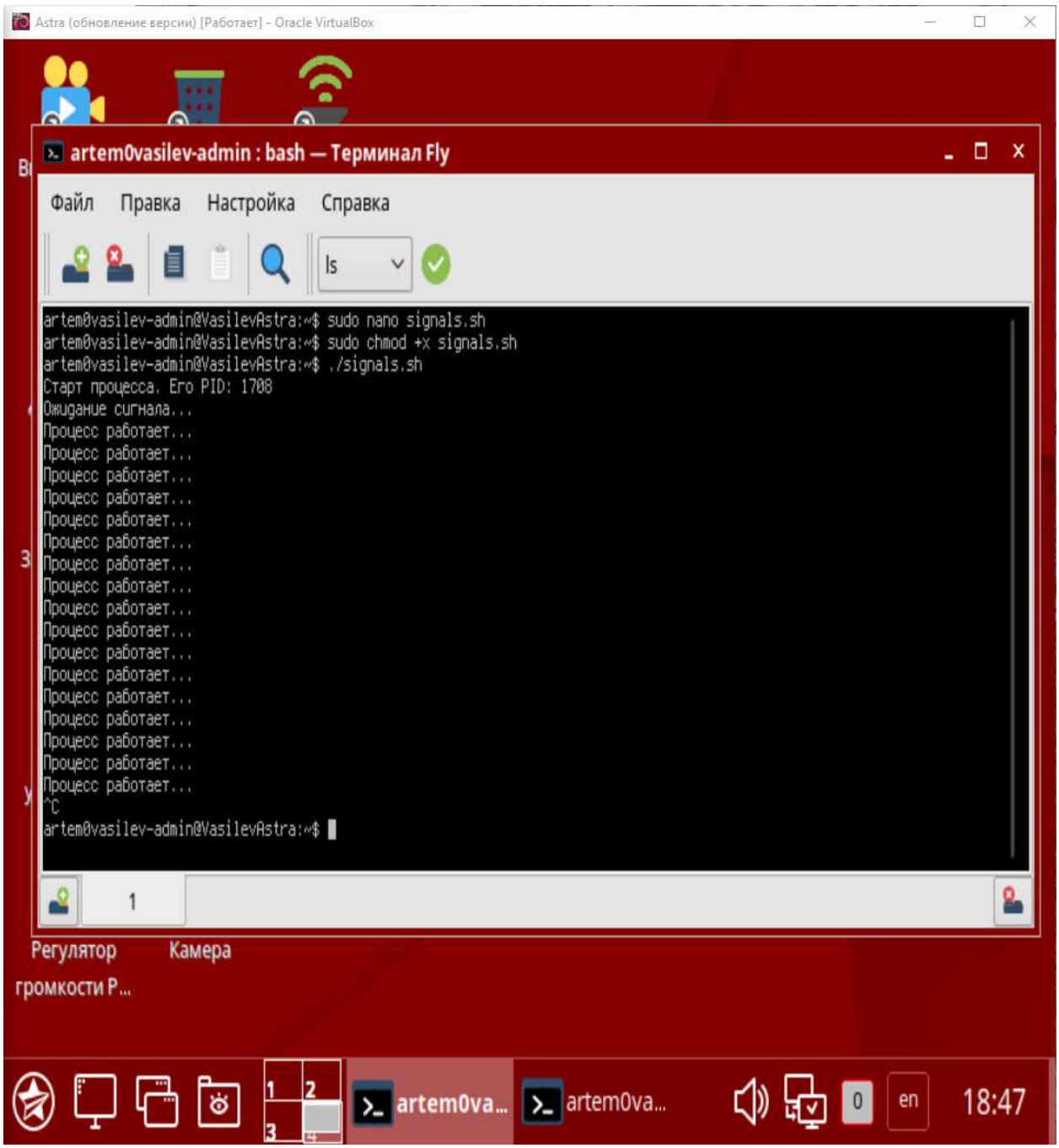

Рисунок 4 – Результат работы скрипта

Среди основных сигналов я выделил следующие:

1. Сигнал SIGTERM. Сигнал запрашивает у процесса корректное завершение работы. Процесс может перехватить этот сигнал, выполнить необходимые действия (сохранение данных, закрытие соединений) и завершиться. Может использоваться в штатном завершении процессов.

2. Сигнал SIGKILL. Сигнал принудительно завершает процесс без возможности обработки. Это единственный сигнал, который не может быть

перехвачен или проигнорирован процессом. Может быть использован, когда процесс не реагирует на SIGTERM.

3. Сигнал SIGINT. Сигнал запрашивает прерывание выполнения интерактивной программы. В большинстве случаев отправляется комбинацией клавиш CTRL+C.

Пример работы данных сигналов можно увидеть на рисунке 5:

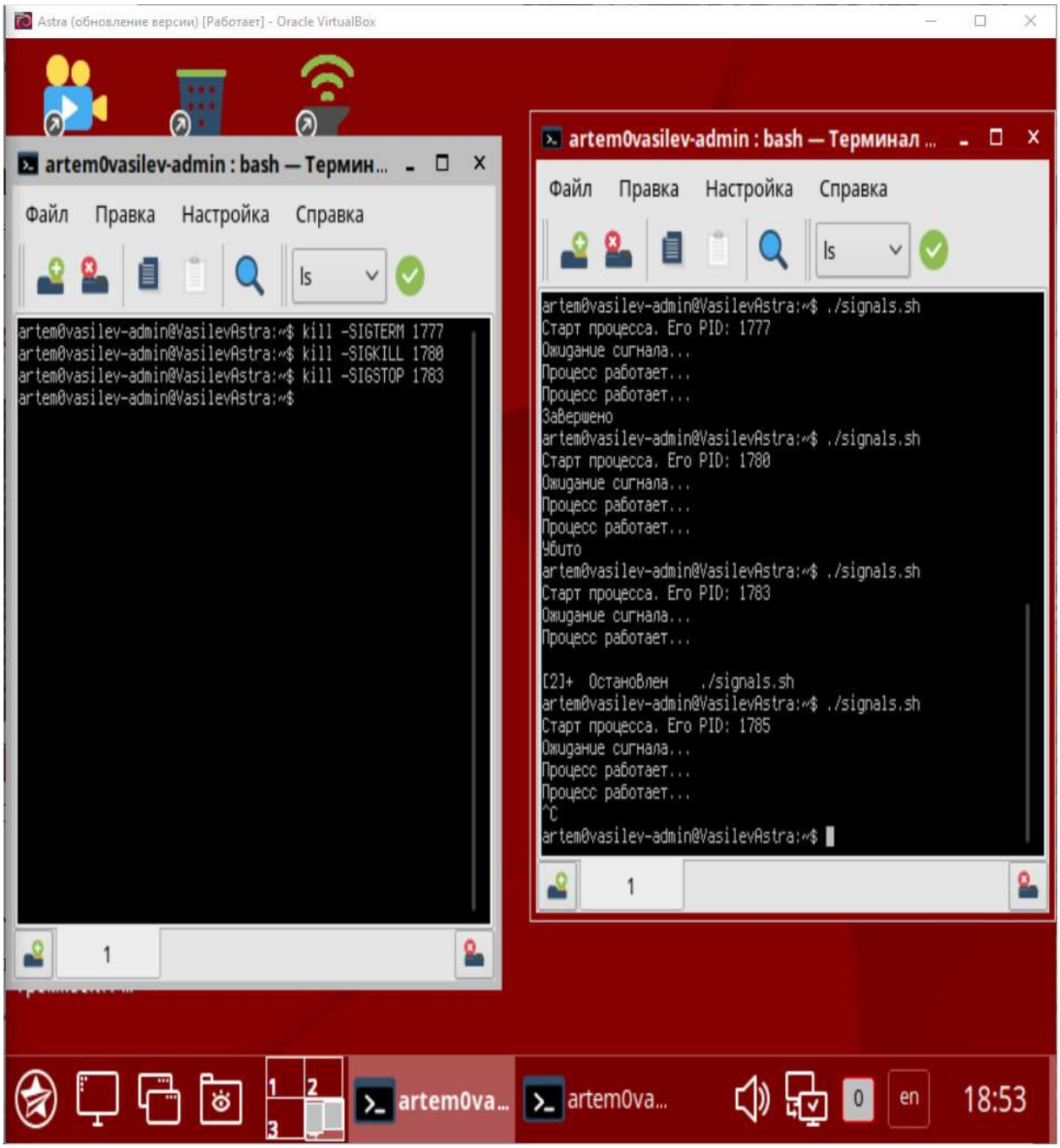

Рисунок 5 – Пример использования сигналов

Отдельно я решил выделить сигналы SIGSTOP и SIGCONT. Сигнал SIGSTOP приостанавливает выполнение процесса, а SIGCONT возобновляет его. Эти сигналы используются для временной остановки процессов без их завершения. Пример использования можно увидеть на рисунке 6:

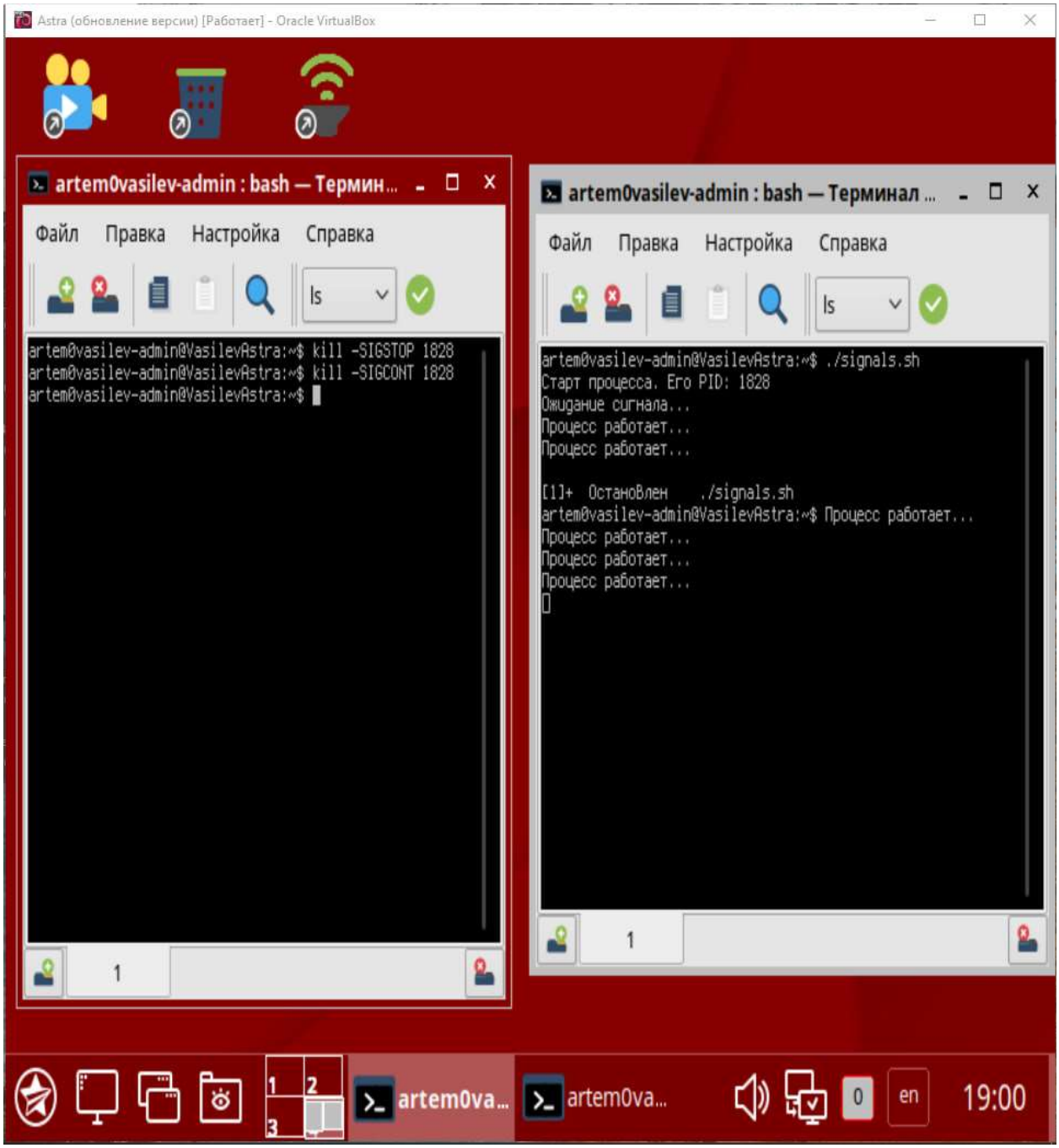

Рисунок 6 – Сигналы SIGSTOP и SIGCONT

2.2. Инструменты командной строки для мониторинга

Linux представляет широкий набор утилит для анализа состояния процессов, потоков, открытых файлов и системных вызовов. Так, команда «ps» используется для получения информации о запущенных процессах в Linux. К примеру, *ps aux* показывает владельца процесса (USER), идентификатор процесса (PID), нагрузка на CPU и память, состояние процесса (R/S/D/T/Z) и другое. Пример использования на рисунке 7:

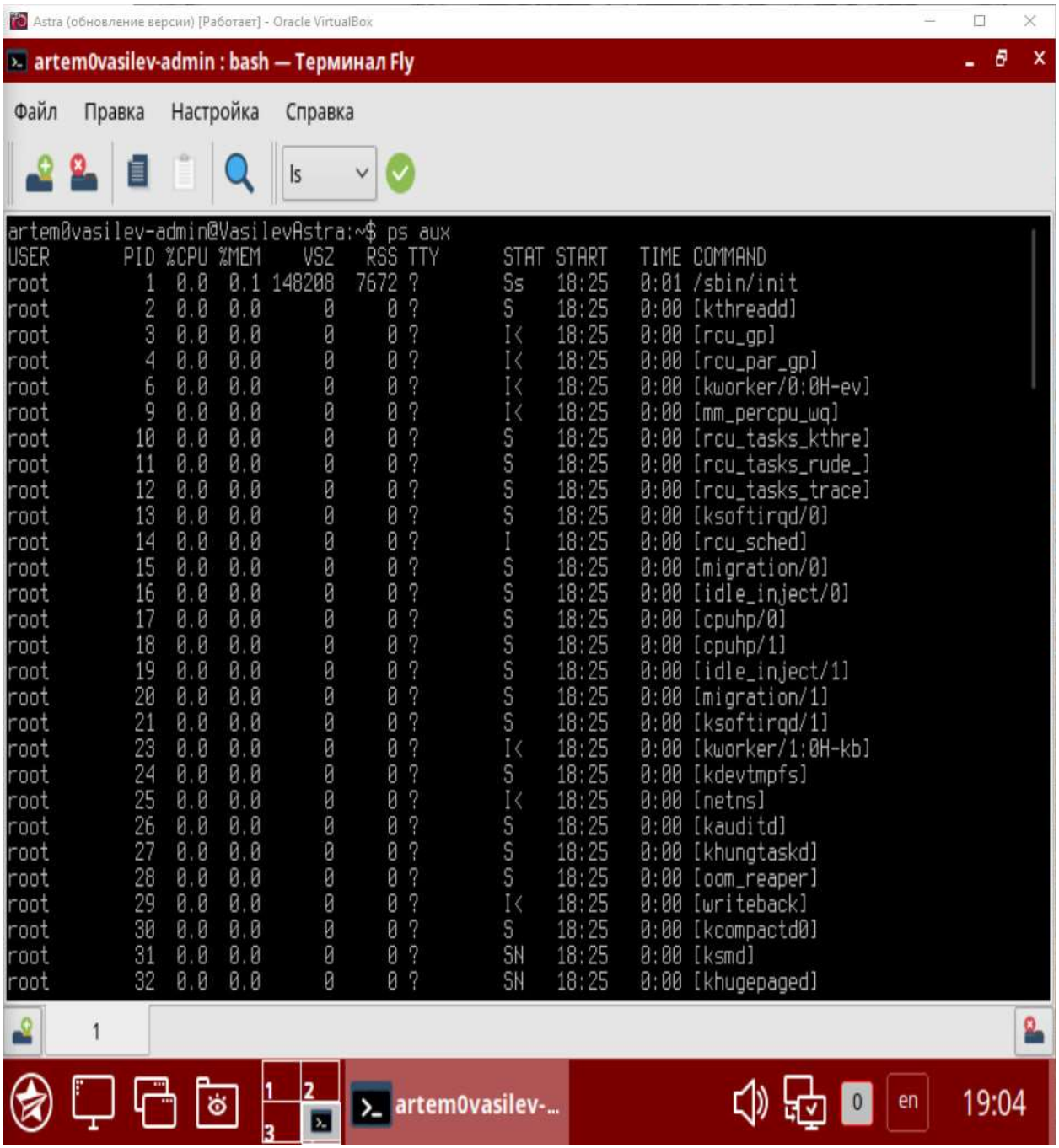

Рисунок 7 – ps aux

В данном случае aux – это ключ. Данная команда может использоваться и с другими ключами, к примеру: -ef (с ключевыми полями UID, PPID, STIME, CMD) или –eLf (с ключевыми полями LWP, NLWP, STAT). Расшифровку всех полей можно посмотреть с помощью команды *man ps*.

Утилиты *top* и *htop* предоставляют динамическую информацию о процессах в реальном времени. Они постоянно обновляют данные, позволяя отслеживать изменения нагрузки системы. Эти инструменты особенно полезны для диагностики проблем производительности и анализа поведения системы под нагрузкой. Так, утилита top показывает состояние по загрузке CPU, использованию памяти, список процессов с сортировкой по CPU и многое другое. Пример использования команды top на рисунке 8:

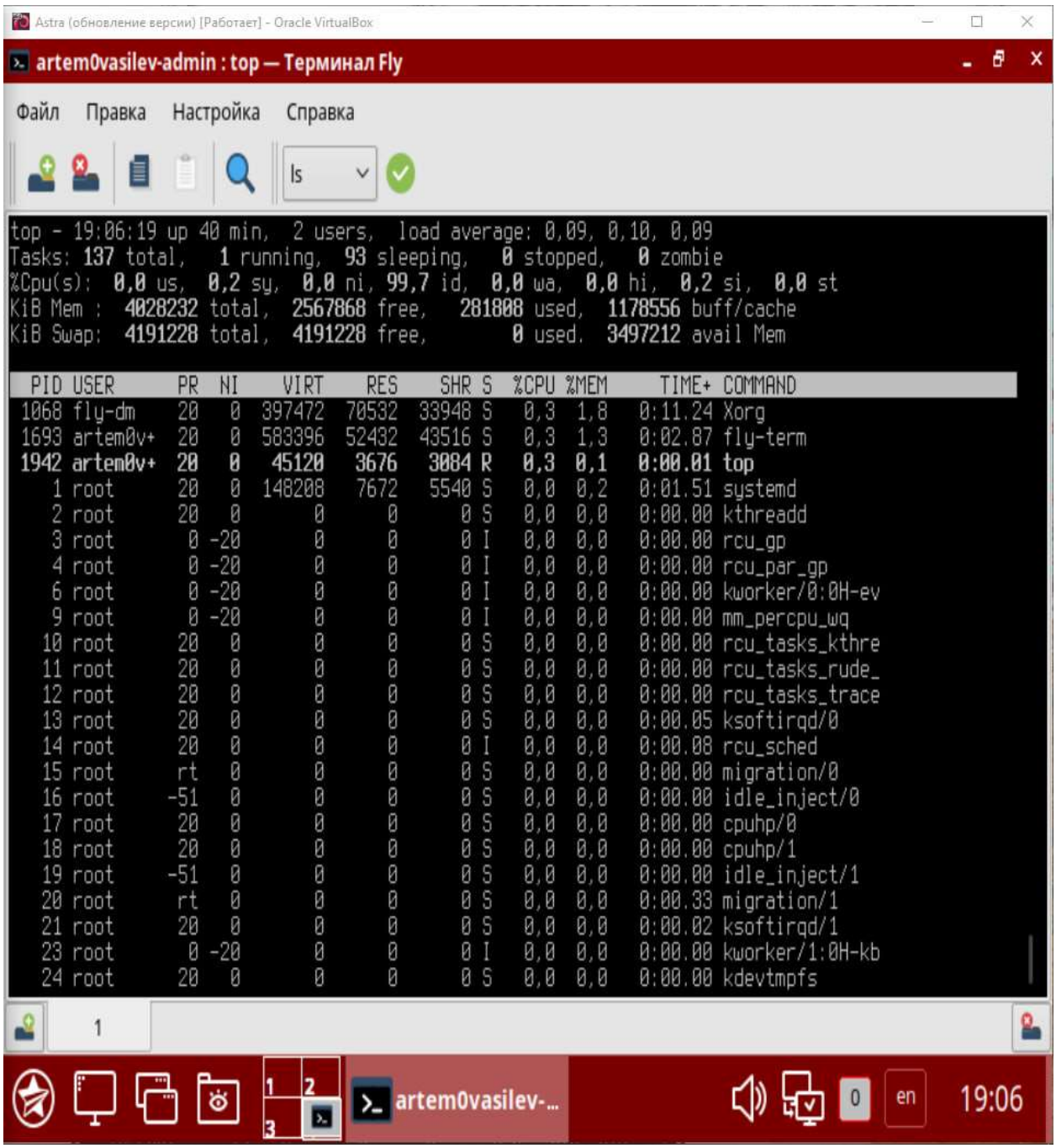

Рисунок 8 – top

Команда *htop* является улучшенной версией *top*. Она имеет цветной интерфейс, поддерживает использование мыши и имеет дополнительные возможности визуализации.

Консольная команда *pstree* используется для того, чтобы увидеть родительские связи между процессами, демонов и сервисов systemd. Простыми словами – для иерархии процессов. Данная команда тоже имеет полезные ключи,

такие как –p (выводить PID), -U(различать пользователей цветом) и другие. Пример использования представлен на рисунке 9:

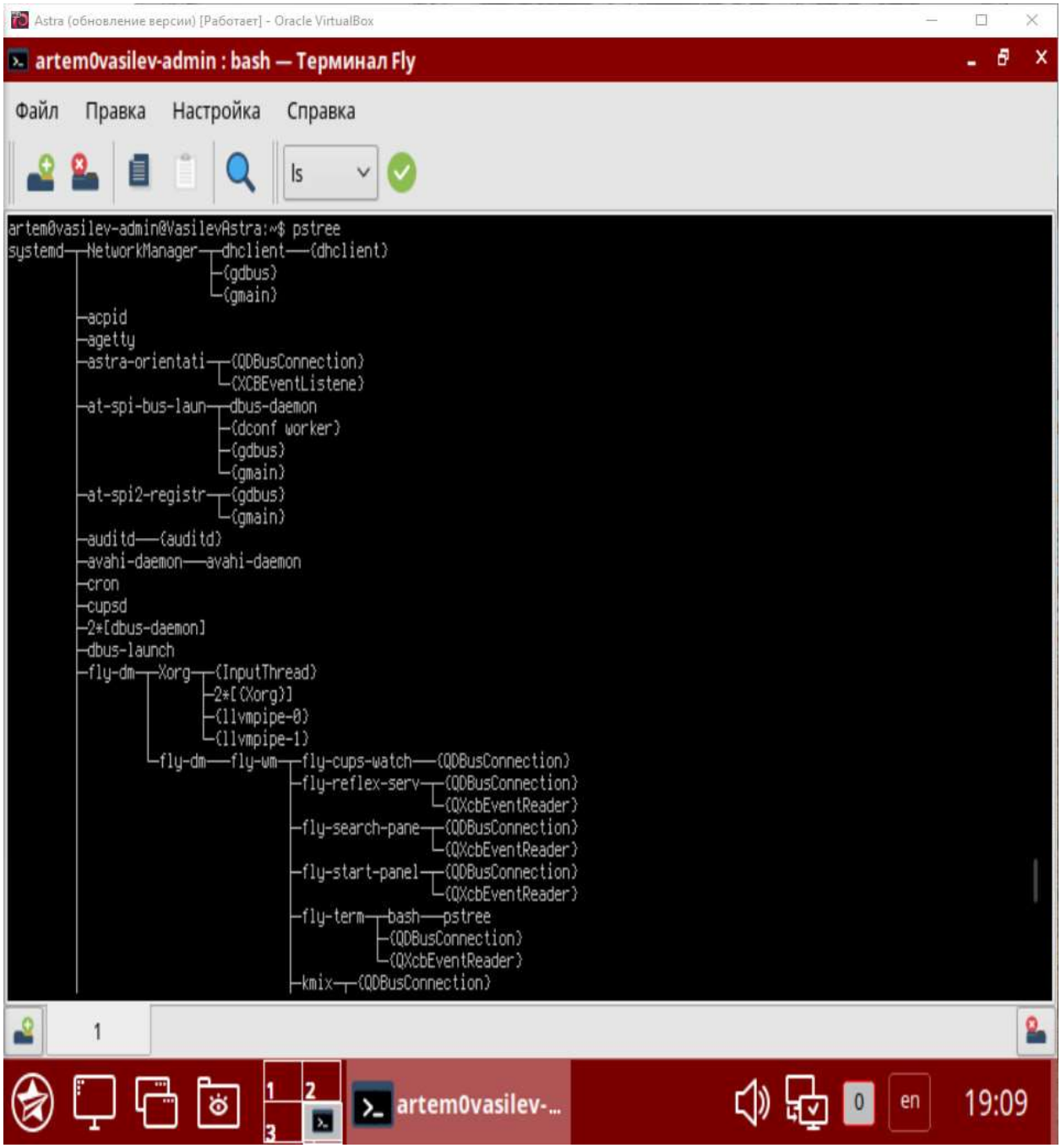

Рисунок 9– pstree

Утилиты *pidof* и *pgrep* используются для быстрого поиска идентификаторов процессов (PID) по их именам. Получается, что они оба ищут PID процесса, но их различие в том, что pidof ищет точное совпадение имени исполняемого файла и возвращает только PID, тогда как pgrep более гибкий, ищет по регулярным выражениям, поддерживает поиск по другим атрибутам

(пользователь, терминал) и может отправлять сигналы, что делает его мощнее для сложных запросов. Пример использования на рисунке 10:

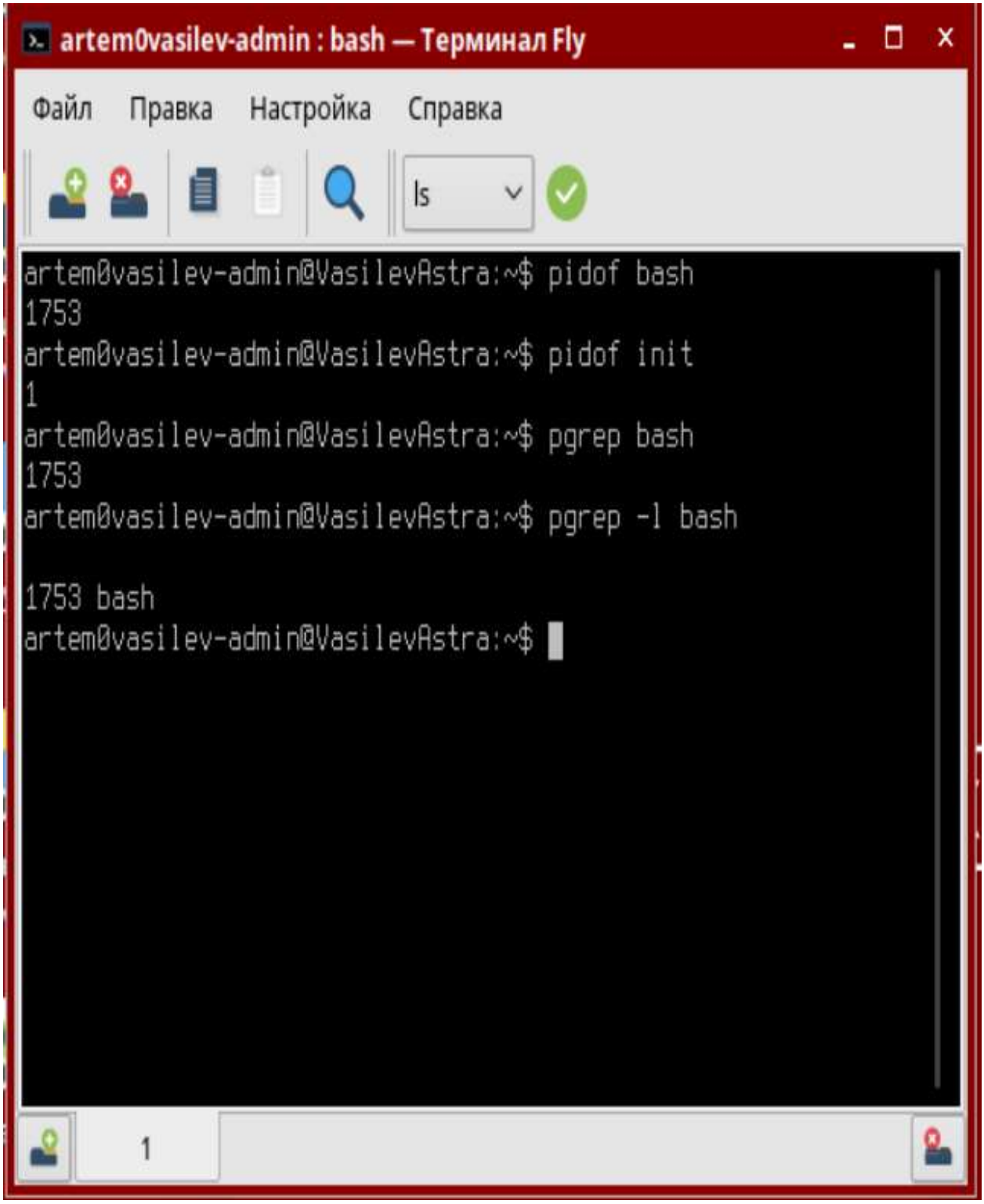

Рисунок 10 – pidof и pgrep

/proc – виртуальная файловая система, представляющая интерфейс к данным ядра. Каждый процесс представлен каталогом /proc/<PID>/. Наиболее

важные файлы: *cat /proc/<PID>/status* – выведет основную информацию (PID, PPID, состояние, память и др.), *cat /proc/<PID>/cmdline* – команда запуска и её аргументы, *cat /proc/<PID>/stat* – показывает состояние процесса (формат ядра), *ls –la /proc/<PID>/fd/* -- показывает открытые файловые дескрипторы (0 – stdin, 1 – stdout, 2 – stderr). Пример использования можно увидеть на рисунке 11 (в данном примере PID у процесса bash = 1696):

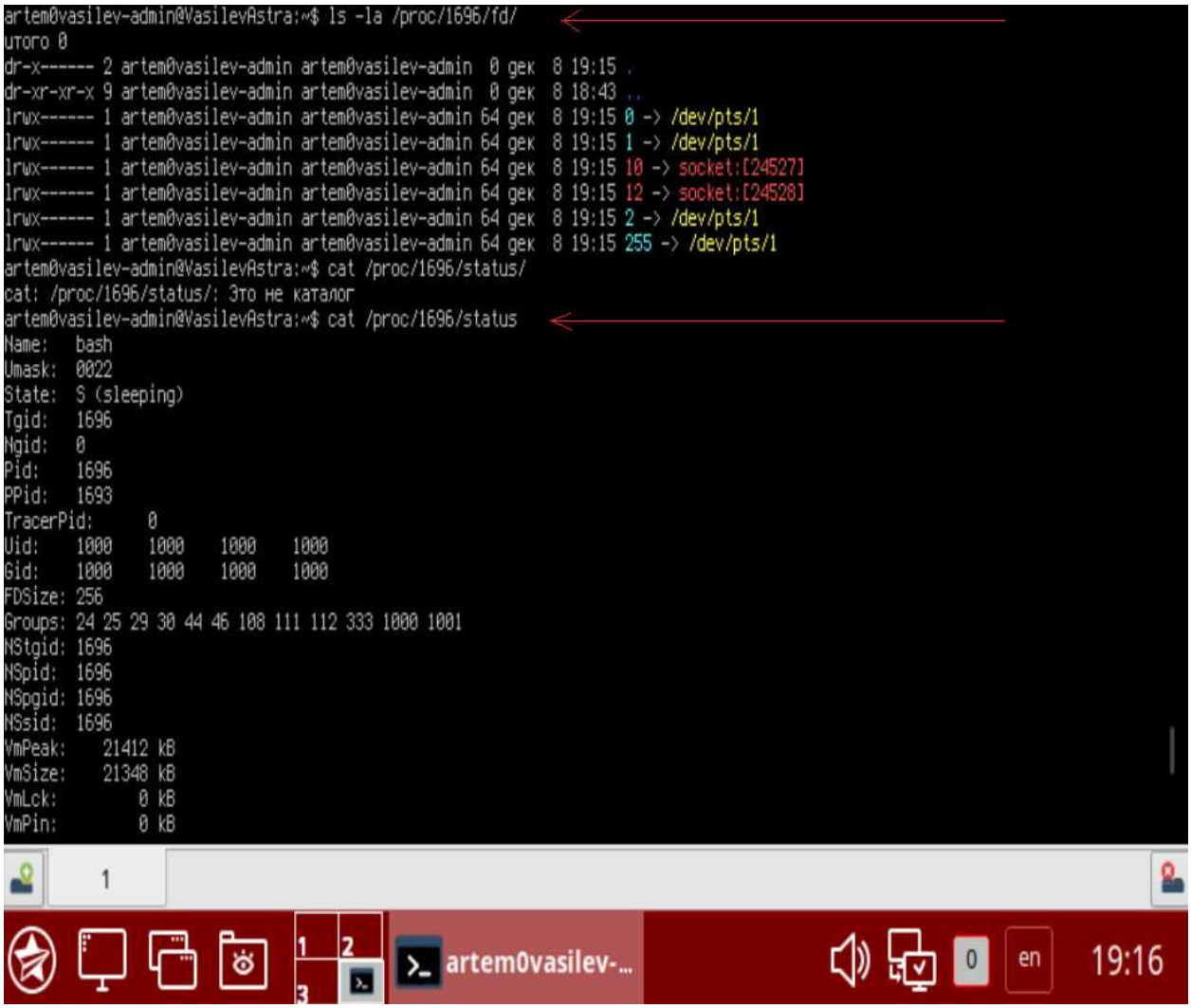

Рисунок 11 – Пример использования /proc

Следующая команда – *lsof*. Она показывает все файлы, открытые процессами, включая обычные файлы, каталоги, сетевые сокеты и устройства. Чтобы посмотреть файлы процесса используется команда *lsof –p <PID>*. Команда, как и многие другие, имеет множество различных ключей, информацию о которых можно прочитать при помощи консольной команды *man lsof*. Пример использования на рисунке 12:

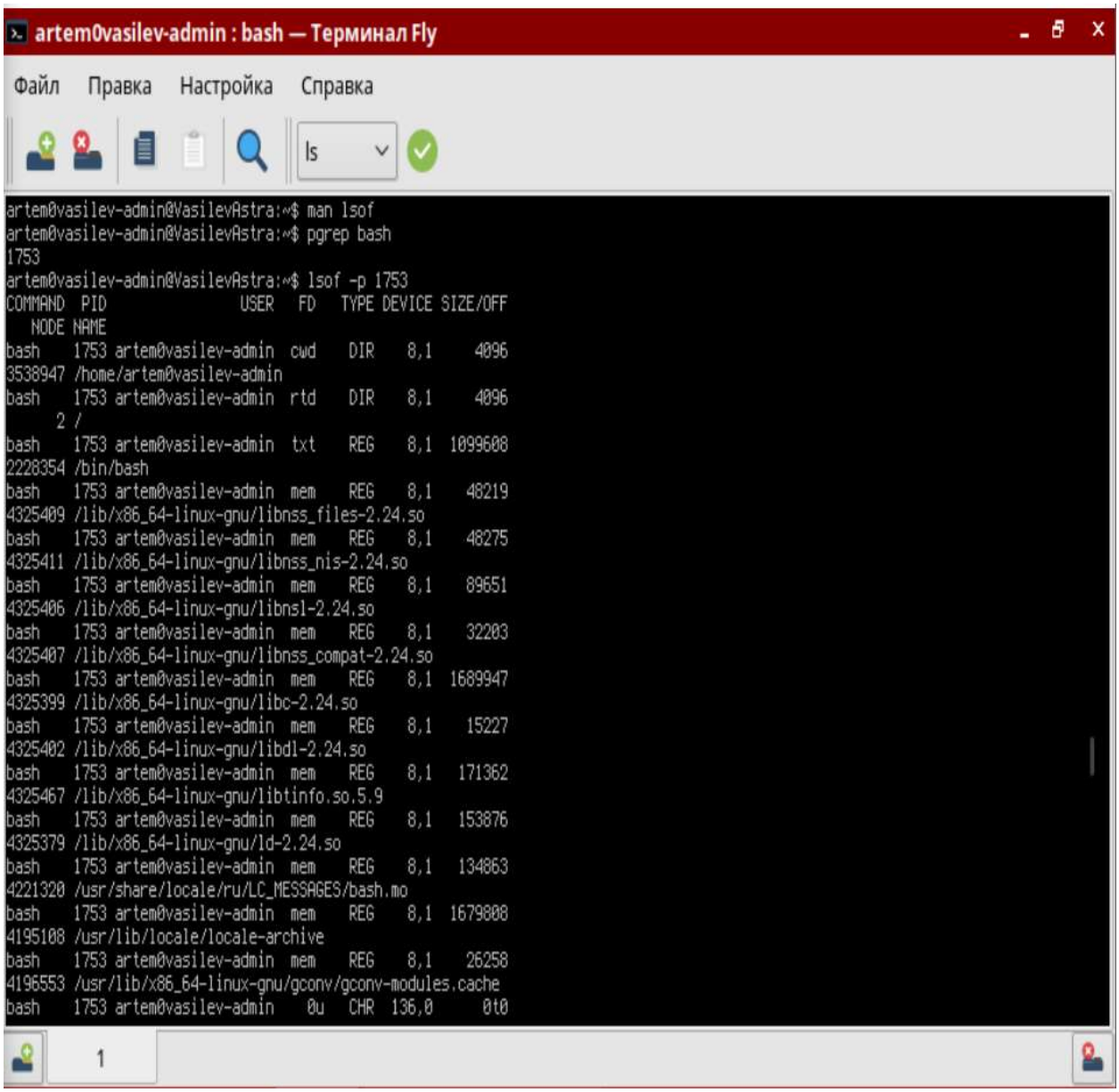

Рисунок 12 – lsof

Утилиты *strace* и *ltrace* используются для отладки и анализа поведения программ. *Strace* трассирует системные вызовы, а *ltrace* – библиотечные вызовы.

Таким образом, инструменты мониторинга предоставляют полный контроль над процессами на разных уровнях абстракции.

2.3. Управление заданиями (Job Control)

Job Control является механизмом командной оболочки, позволяющим пользователю управлять несколькими процессами (заданиями) в рамках одного сеанса работы. Он позволяет запускать задания в фоне, приостанавливать их, менять на передний план и так далее.

Некоторые команды shell занимают много времени при выполнении. Эти команды можно запустить в фоновом режиме с использованием &, освобождая тем самым терминал для других задач. Общий формат для запуска команд в фоновом режиме следующий: <*command*> &. Таким образом, процесс выполняется в фоне, а пользователь может вводить новые команды. Фоновый процесс получает отдельный job ID и PID, что пользовает им управлять отдельно от текущей сессии терминала. Комбинация клавиш Ctrl+Z отправляет текущему процессу на переднем плане сигнал, который приостанавливает его выполнение и переводит в состояние остановки. Пример запуска процессов, просмотр *jobs*, а также остановка процесса представлены на рисунке 13:

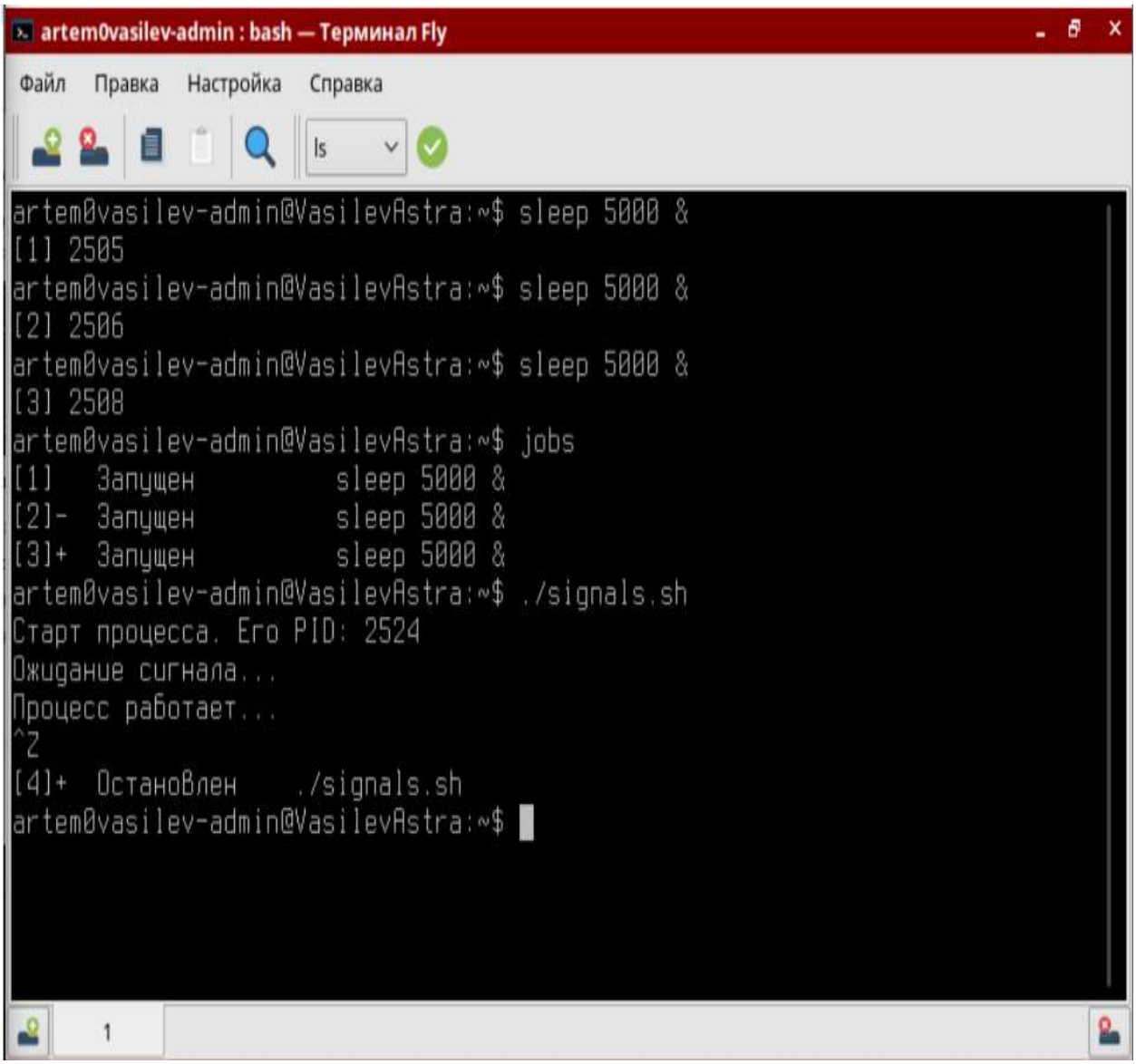

Рисунок 13 – Управление заданиями

Кроме *jobs* существуют и другие команды по работе с заданиями, к примеру команда *fg* возвращает задание на передний план, возобновляя его выполнение, если оно было приостановлено. Команда *bg* возобновляет выполнение приостановленного задания, но в фоновом режиме. Пример на рисунке 14:

```
artem0vasilev-admin@VasilevAstra:~$ sleep 5001 &
[1] 2742
artem0vasilev-admin@VasilevAstra:~$ sleep 5002 &
[2] 2743
artem0vasilev-admin@VasilevAstra:~$ sleep 5003 &
[3] 2745
artem0vasilev-admin@VasilevAstra:~$ jobs
[1]   Запущен         sleep 5001 &
[2]-  Запущен         sleep 5002 &
[3]+  Запущен         sleep 5003 &
artem0vasilev-admin@VasilevAstra:~$ fg %1
sleep 5001
^Z
[1]+  Остановлен    sleep 5001
artem0vasilev-admin@VasilevAstra:~$ bg %1
[1]+ sleep 5001 &
artem0vasilev-admin@VasilevAstra:~$ jobs
[1]   Запущен         sleep 5001 &
[2]-  Запущен         sleep 5002 &
[3]+  Запущен         sleep 5003 &
artem0vasilev-admin@VasilevAstra:~$
```

Рисунок 14 – fg и bg

Что делать если необходимо продолжать выполнения задания после выхода из терминала? На помощь приходит команда *disown*, которая убирает задачу из списка job control, вследствие чего процесс не завершается при закрытии терминала.

3. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

3.1. Эксперимент с nice и renice

В Linux каждый процесс имеет параметр *nice*, который влияет на его приоритет при планировании. Диапазон значений: от -20 (наивысший приоритет) до 19 (наименьший приоритет), а стандартное значение – 0. Чем больше значение *nice*, тем меньше CPU он получает. Изменение *nice* не гарантирует точного процента CPU, но влияет на распределение времени между конкурентными задачами.

Целью данного эксперимента является изучение влияния приоритета процесса на распределение процессорного времени. Для этого используются механизмы *nice* и *renice*, которые позволяют изменять относительный приоритет задач, планируемых стандартным планировщиком Linux. Синтаксис *nice* выглядит следующим образом: *nice –n <приоритет> <команда>*.

Создадам для эксперимента две задачи, загружающие процессор постоянными вычислениями. Первая задача в первом терминале с обычным приоритетом, т.е. по стандарту 0. В команде *yes > /dev/null*, yes – используется для бесконечной генерации вывода, а перенаправление в /dev/null устраняет операцию вывода на экран. (рисунок 15):

```
artem0vasilev-admin@VasilevAstra:~$ yes > /dev/null
```

Рисунок 15 – Запуск первой задачи

Во другом терминале запускаю вторую задачу с пониженным приоритетом, к примеру 15 (рисунок 16):

```
artem0vasilev-admin@VasilevAstra:~$ nice -n 15 yes > /dev/null
```

Рисунок 16 – Запуск второй задачи

Таким образом, в системе запущены 2 процесса yes, но они имеют разные приоритеты. Проанализирую выделение процессорного времени с помощью утилиты *top* (рисунок 17):

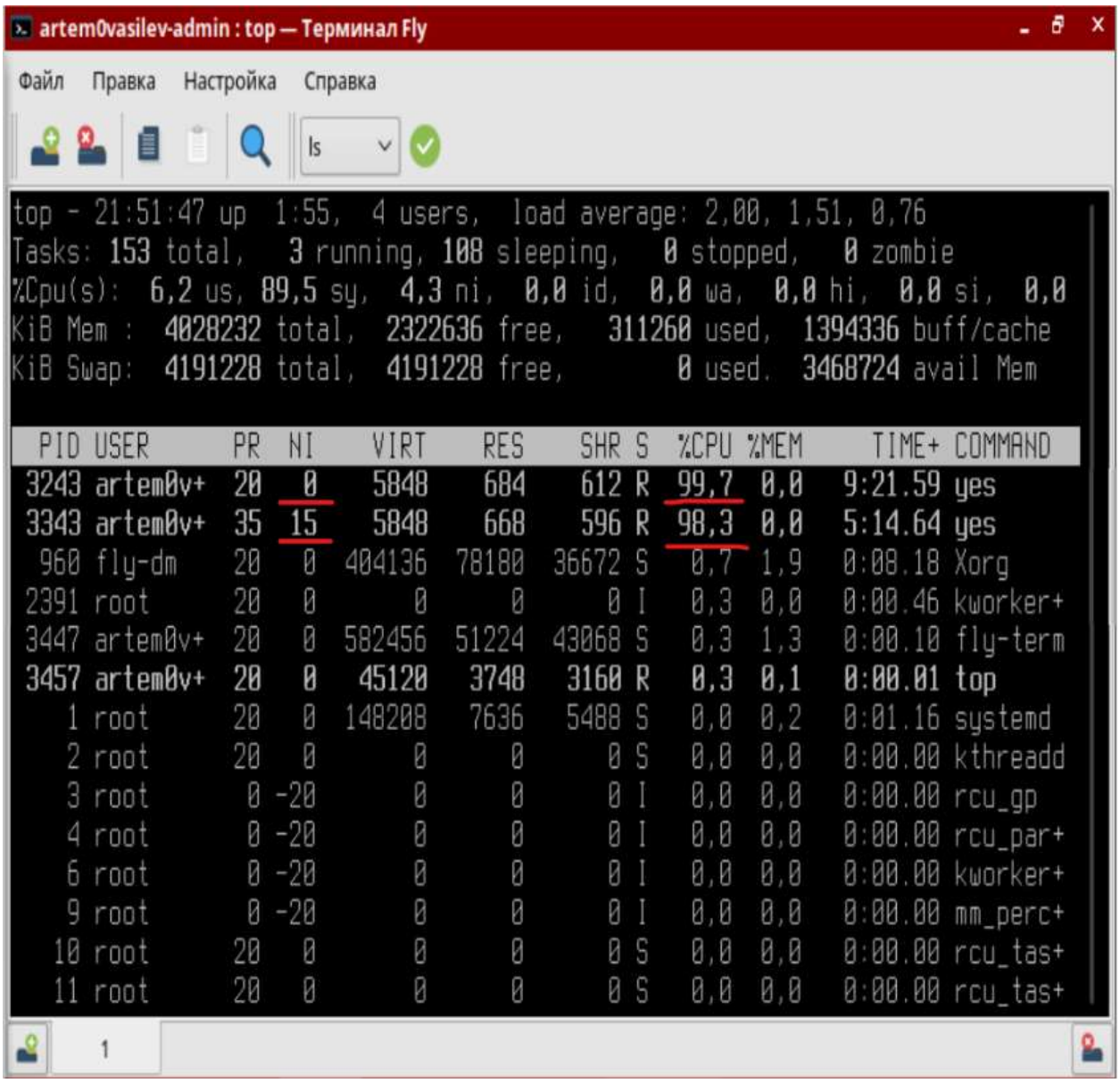

Рисунок 17 – Сравнение процессов

На рисунке 17 наглядно видно, что процесс с более высоким приоритетом (обозначается NI), а именно 0, получает больше процессорного времени (%CPU), чем процесс с приоритетом 15.

С помощью команды *renice*, которая имеет синтаксис *renice <приоритет> -p <PID>*, можно изменить приоритет уже запущенного процесса. Изменю приоритет процесса с 15 на -5 и зафиксирую изменения (рисунки 18-19):

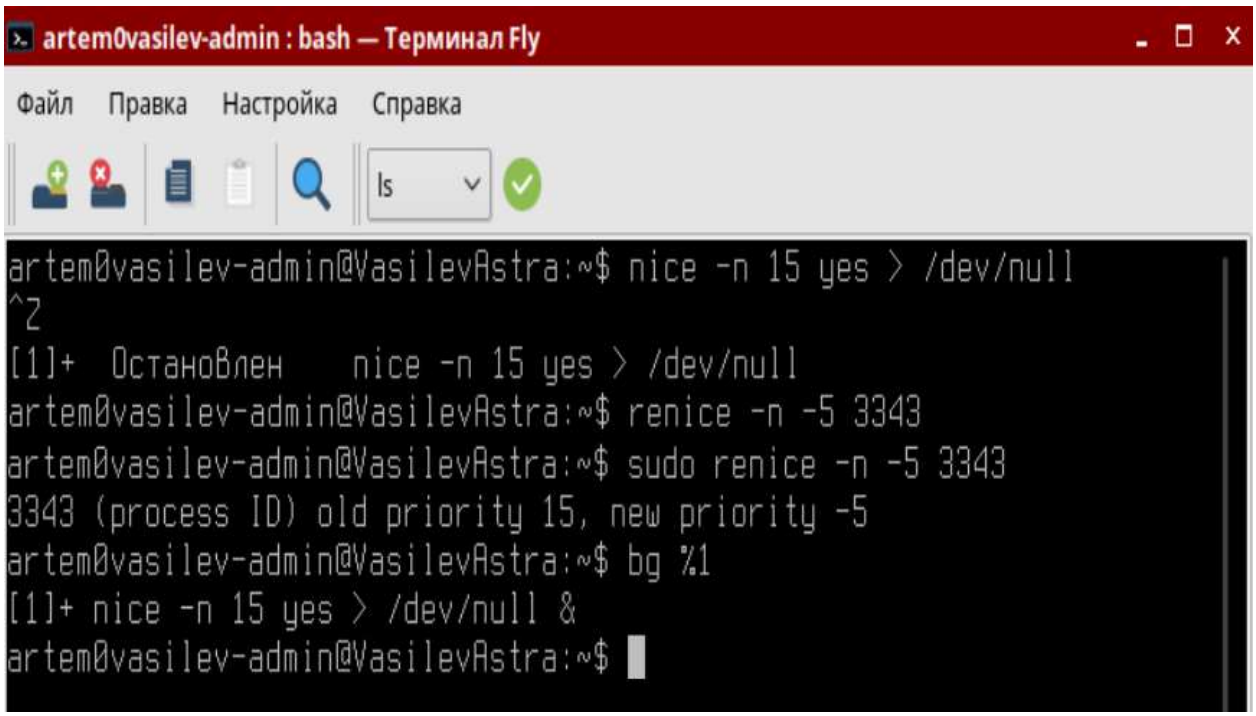

Рисунок 18 – Изменение приоритета (1)

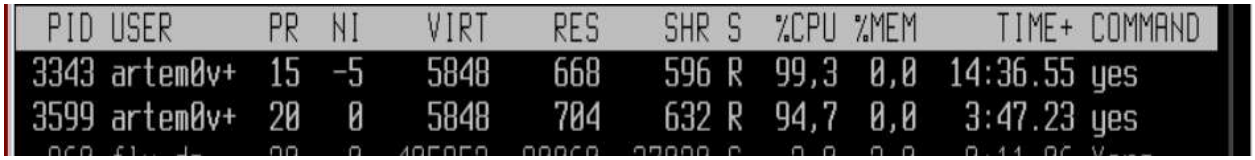

Рисунок 19 – Изменение приоритета (2)

Действительно, приоритет процесса изменился с 15 на -5, и теперь он стал потреблять больше процессорного времени, чем процесс, со стандартным приоритетом 0.

3.2. Анализ процесса через /proc

Целью эксперимента является практическое изучение структуры виртуальной файловой системы */proc* и анализ состояния конкретного процесса, а именно демона. Для того, чтобы выбрать процесс, являющийся демоном, введу команду *ps aux* и найду процесс, который работает от root (UID = 0) и имеет TTY= ? (т.е. процесс не привязан к терминалу). Так мой выбор пал на процесс watchdogd – процесс, который работает в фоновом режиме и следит за здоровьем системы, выполняет проверки и может перезагрузить при зависании или сбое. PID процесса – 81. (рисунок 20):

Рисунок 20 – Выбор процесса watchdogd

Посмотрю обобщённую информацию о процессе с помощью *cat /proc/81/status*. Здесь можно увидеть: PID и PPID, состояние процесса – S (sleeping), что означает состояние сна, количество потоков threads =1 и другое (рисунок 21):

```
artem0vasilev-admin@VasilevAstra:~$ cat /proc/81/status
Name:   watchdogd
Umask:  0000
State:  S (sleeping)
Tgid:   81
Ngid:   0
Pid:    81
PPid:   2
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups:
NStgid: 81
NSpid:  81
NSpgid: 0
NSsid:  0
Threads:        1
SigQ:   0/7525
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: ffffffffffffffff
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 000001ffffffffff
CapEff: 000001ffffffffff
CapBnd: 000001ffffffffff
```

Рисунок 21 – Основная информация о процессе

Файл */proc/81stat* содержит полную статистику процесса в виде 52 числовых и текстовых полей в одной строке, включая идентификаторы (PID, PPID), состояние выполнения, приоритеты, время использования CPU в тиках ядра и многое другое. Файл */proc/81/statm* предоставляет информацию об использовании памяти процессом в виде семи чисел, представляющих различные аспекты распределения виртуальной и физической памяти: общий размер образа процесса, резидентный набор (реально используемая память), разделяемые страницы, а также сегменты кода, данных и стека. Можно увидеть результат на рисунке 22:

```
artem0vasilev-admin@VasilevAstra:~$ cat /proc/81/stat
81 (watchdogd) S 2 0 0 0 -1 2129984 0 0 0 0 0 0 0 0 -51 0 1 0 20 0 0 18446744073709551615 0
0 0 0 0 0 0 2147483647 0 0 0 0 17 0 50 1 0 0 0 0 0 0 0 0 0 0 0 0
artem0vasilev-admin@VasilevAstra:~$ cat /proc/81/statm
0 0 0 0 0 0 0
artem0vasilev-admin@VasilevAstra:~$
```

Рисунок 22 – stat и statm

Следующая команда *ls –l /proc/81/fd* (рисунок 23). Каталог fd содержит символические ссылки на открытые файловые дескрипторы. «Итого 0» означает, что в каталоге нет файлов, занимающих место. Следующие 2 строки обозначают текущую директорию (в конце стоит «.») /proc/81/fd и родительскую (в конце стоит «..») /proc/81 соответственно. «dr-xr-xr-x» и «dr-x-----» – это строки, обозначающие права доступа к файлу или директории.

```
artem0vasilev-admin@VasilevAstra:~$ sudo ls -la /proc/81/fd
итого 0
dr-x------ 2 root root 0 дек 13 23:43 .
dr-xr-xr-x 9 root root 0 дек 13 23:35 ..
```

Рисунок 23 – Каталог fd

Посмотрю потоки процесса, если они есть с помощью команды *ls –ld /proc/81/task*. У этого процесса есть как минимум 1 поток, потому что количество ссылок = 3. Загляну во внутрь с помощью команды *ls –la /proc/81/task*. Вижу следующие элементы: текущая директория, родительская директория, а также поддиректория, представляющая поток процесса. Каждый отдельный каталог = отдельный поток (TID) (рисунок 24):

```
artem0vasilev-admin@VasilevAstra:~$ ls -ld /proc/81/task
dr-xr-xr-x 3 root root 0 дек 13 23:43 /proc/81/task
artem0vasilev-admin@VasilevAstra:~$ ls -la /proc/81/task/
итого 0
dr-xr-xr-x 3 root root 0 дек 13 23:43 .
dr-xr-xr-x 9 root root 0 дек 13 23:35 ..
dr-xr-xr-x 7 root root 0 дек 14 00:02 81
```

Рисунок 24 – Потоки процесса

Можно подвести небольшой итог о том, что файловая система /proc является виртуальной и предоставляет интерфейс для получения информации о

процессах напрямую от ядра Linux. Анализ каталога */proc/<PID>* позволяет изучить состояние процесса, его параметры планирования, использование памяти и многое другое.

3.2. Анализ системы при помощи утилиты atop

Atop — это мощный интерактивный монитор системных ресурсов для Linux-систем, который предоставляет детальную информацию о загрузке процессора, использовании памяти, активности дисков, сетевой нагрузке и работе отдельных процессов. В отличие от более простых утилит типа top или htop, atop обладает уникальной особенностью — возможностью логирования системных метрик с последующим анализом исторических данных.

Интерфейс atop разделен на две основные части, что видно на рисунке 25:

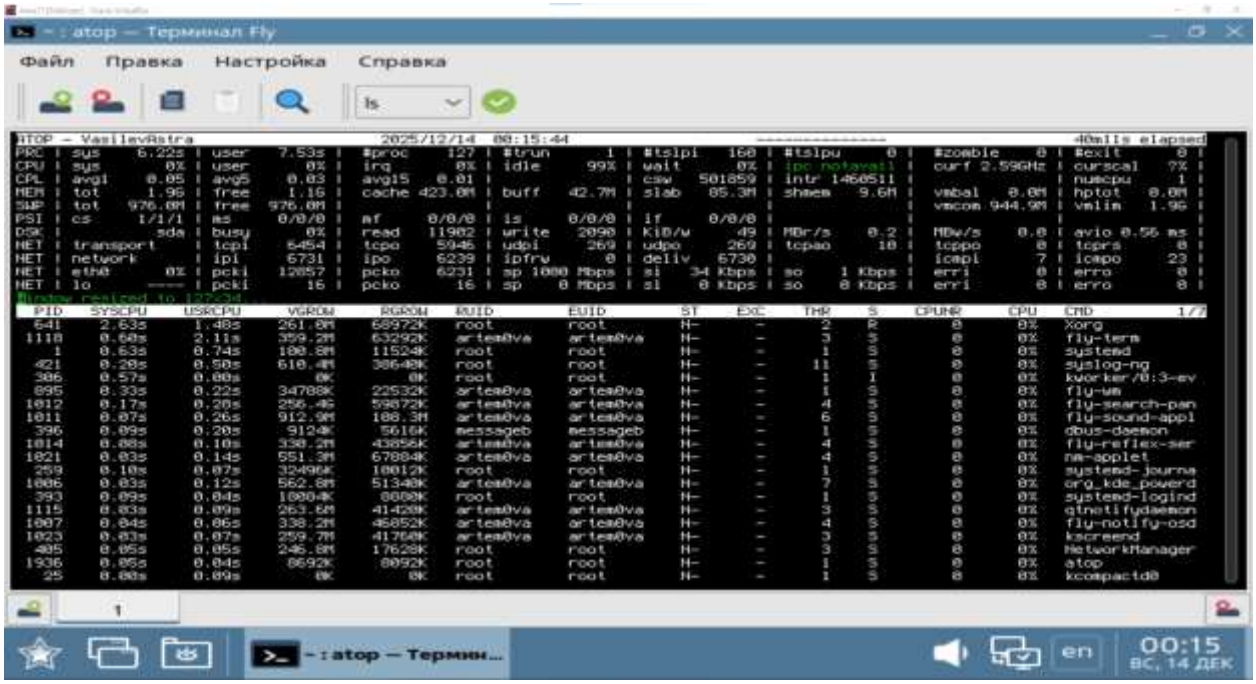

Рисунок 25 – Утилита atop

На рисунке можно увидеть следующие метрики: PRC (Process) – общая информация о процессах, CPU (загрузка процессора), MEM (использование памяти) и другое.

Снизу отображаются активные процессы, отсортированные по потреблению ресурсов.

В отличие от top, atop способен отображать ресурсы, потреблённые процессом за интервал времени, а не только текущее состояние.

При нажатии клавиши «h» появляется окно с основным списком доступных команд (рисунок 26). Среди них сортировка процессов («c» – по CPU, m – по памяти и другие), фильтрация («p» – ввести PID или имя), режимы отображения («g» – общая информация, «d» – детали по дискам), а также управления логами («t» – следующий снимок, «T» – предыдущий снимок и так далее). Чтобы выйти из atop необходимо нажать клавишу «q».

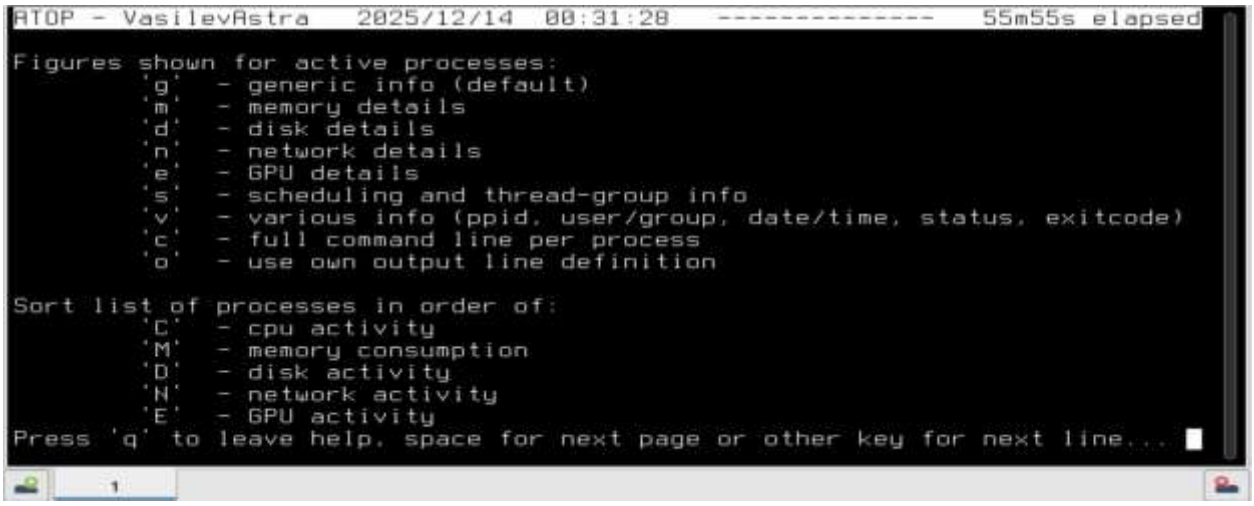

Рисунок 26 – Список основных команд в atop

Одним из ключевых преимуществ atop является поддержка исторического анализа. Утилита может записывать статистику в файлы журналов, расположенные в каталоге */var/log/atop*, представленном на рисунке 27:

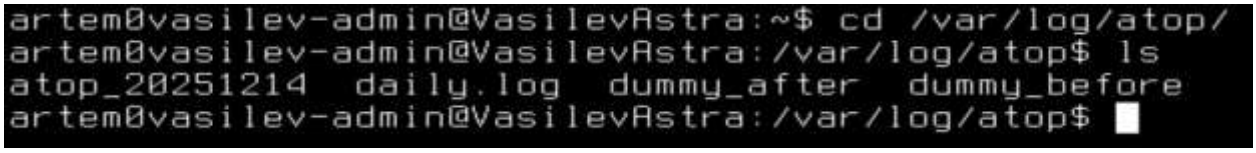

Рисунок 27 – Каталог atop-а

Файлы формата *atop_** являются бинарными файлами, содержащими снимки системы через равные промежутки времени. Они записаны в формате *atop_YYYYMMDD*. Такие файлы могут пригодиться при расследовании инцидентов после их возникновения (например, почему какое-то приложение работало медленно в определённое время и многое многое другое), то есть можно смотреть нагрузку в определённое время и анализировать её. Для просмотра раннее сохранённых данных используется команда *atop -r /var/log/atop/atop_**.

Таким образом, утилита atop является мощным инструментом мониторинга и анализа состояния системы Linux. В отличие от стандартных средств, она предоставляет расширенные возможности по анализу ресурсов и поддерживает работу с историческими данными. Использование atop позволяет глубоко исследовать поведение процессов и эффективно диагностировать проблемы производительности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были изучены теоретические основы и практические механизмы управления процессами в операционной системе Astra Linux Special Edition. Рассмотрены понятия процессов и потоков, их жизненный цикл, а также принципы работы планировщика задач.

В практической части были исследованы способы взаимодействия с процессами при помощи сигналов, средств мониторинга и управления заданиями оболочки. Экспериментальная часть позволила на практике оценить влияние приоритетов процессов с помощью nice и renice на распределение процессорного времени, а также изучить состояние процессов через файловую систему /proc и возможности утилиты atop. Поставленная цель курсовой работы была достигнута, а все задачи успешно решены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Все, что вам нужно знать о процессах в Linux // Rus-Linux URL: https://rus-linux.net/MyLDP/proc/processes-in-Linux.html (дата обращения: 10.12.2025).

2. Сегментация памяти // Хабр URL: https://habr.com/ru/articles/345766/ (дата обращения: 10.12.2025).

3. Изменение приоритета процесса // РЕД ОС URL: https://redos.red-soft.ru/base/redos-7_3/7_3-administation/7_3-processes/7_3-change-priority-process/ (дата обращения: 11.12.2025).

4. Atop // РЕД ОС URL: https://redos.red-soft.ru/base/redos-8_0/8_0-administation/8_0-processes/8_0-monitoring-proc/8_0-dynamic-monitoring-proc/8_0-atop/?sphrase_id=1378413 (дата обращения: 11.12.2025).

5. Управление Заданиями // Википедия URL: https://ru.wikipedia.org/wiki/Управление_заданиями (дата обращения: 11.12.2025).

6. Официальная документация в терминале Linux: man ps, man fork, man nice, man renice, man top, man atop, man proc.